

North South University



Project Report

Name - Mohammad Mahabub A Mostafa Badhan

ID - 2616185050

Section - 01

Course - CSE534

Faculty - Safat Siddiqui Sir

Intelligent Test Execution and Reporting Framework

1. Introduction

Software testing is an essential component of software engineering to measure the quality of software, find defects, and ensure reliability before deployment. Manual testing is a tedious, repetitive, and error-prone process in today's software development, and thus automated testing is a necessity. Automated testing frameworks help developers to run tests in a more efficient way, analyze failures, and improve the maintainability of software.

This project, Intelligent Test Execution and Reporting Framework, is about implementing different levels of testing, like unit testing, integration testing, and system testing, in one automated framework. The framework also provides result analysis, flaky test detection, graphical visualization, and a web-based dashboard for interactive monitoring.

2. Project Objectives

The main objectives of this project are:

- To implement automated software testing techniques
- To perform unit, integration, and system testing
- To develop a custom test execution engine
- To generate automated test reports
- To visualize testing results graphically
- To detect flaky or unstable tests
- To provide a web dashboard for monitoring test execution

3. Technologies Used

Technology	Purpose
Python	Core programming language
pytest	Automated testing
Flask	Web dashboard
Matplotlib	Graph visualization
GitHub	Version control and repository hosting

4. System Architecture

The project follows a modular architecture where different testing components interact with the execution framework.

Main Components

1. Source Modules
2. Unit Tests
3. Integration Tests
4. System Tests
5. Test Runner Engine
6. Visualization Module
7. Dashboard Interface
8. Reporting System

5. Project Implementation

5.1 Source Modules

The project contains several source modules, such as:

- Calculator module
- User service
- Order service
- System workflow module

These modules simulate a small software system that can be tested at multiple levels.

5.2 Unit Testing

Unit testing was implemented to validate individual functions independently. Small functional units such as arithmetic operations and user creation logic were tested using pytest.

Examples:

- Addition operation testing
- Division validation
- Empty user validation
- Invalid order checking

Unit testing ensured that each component behaved correctly in isolation.

5.3 Integration Testing

Integration testing was implemented to verify communication between multiple modules. The project integrated the user service with the order service to validate data flow between components.

The integration tests checked:

- User creation during order placement
- Correct item assignment
- Validation handling across modules

This testing phase ensured that combined modules functioned properly together.

5.4 System Testing

System testing validated the complete application workflow from start to finish. A full order placement process was simulated to verify the behavior of the entire system.

The workflow included:

1. User creation
2. Order generation
3. Payment simulation
4. Delivery processing
5. Success response generation

System testing confirmed that the application functioned correctly as a complete system.

5.5 Custom Test Execution Engine

A custom test execution engine was developed using Python subprocess handling. Instead of manually executing pytest commands, the framework automated test execution and generated summarized output.

The engine performs:

- Automated test execution
- Pass/fail analysis
- Report generation
- Flaky test analysis

This feature transformed the project from simple testing scripts into a reusable testing framework.

5.6 Flaky Test Detection

An advanced flaky test detection mechanism was implemented to identify unstable tests. The framework executes the same tests multiple times and compares the results.

If test outcomes differ between executions, the framework marks them as flaky.

This feature improves reliability because flaky tests can produce inconsistent software quality measurements.

5.7 Visualization Module

A graphical visualization system was implemented using Matplotlib. Test results are displayed as bar charts showing:

- Passed tests
- Failed tests

Visualization improves readability and provides a clearer understanding of software quality status.

5.8 Web Dashboard

A web-based dashboard was developed using Flask to create an interactive user interface for the framework.

Dashboard features include:

- Run Tests button
- Real-time test execution status
- Auto-refresh every 60 seconds
- Live raw test logs
- Graph visualization
- Result summary display

The dashboard significantly improved usability and demonstrated practical software engineering concepts.

6. Findings of the Project

The project successfully demonstrated that automated testing frameworks can improve software quality and simplify test management.

Several important findings were observed:

- Automated testing reduces repetitive manual work
- Layered testing improves defect detection capability
- Integration testing is essential for module interaction validation
- System testing ensures complete workflow correctness
- Visualization improves result interpretation
- Flaky test detection increases reliability
- Dashboard interfaces improve monitoring efficiency

The project also demonstrated how combining testing automation with reporting and visualization creates a more effective software quality assurance process.

7. Difficulties Faced During Development

Several technical and implementation challenges were encountered during the project development process.

7.1 Import and Module Errors

Initially, Python module import issues occurred due to incorrect project structure and missing `__init__.py` files.

Solution

The project folders were converted into Python packages by adding appropriate initialization files and correcting import paths.

7.2 Pytest Installation and Environment Issues

The pytest command was initially not recognized in the Windows PowerShell environment.

Solution

Pytest was executed using:

</> Bash

```
python -m pytest
```

This resolved environment path issues.

7.3 Integration Test Failures After Refactoring

During development, function names were modified while improving system integration, causing test failures.

Solution

The affected test cases were updated to align with the modified implementation.

7.4 Unicode Encoding Errors

Unicode errors occurred while printing emoji characters inside the Windows terminal and Flask subprocess environment.

Solution

Unsupported Unicode symbols were removed and replaced with standard text messages to ensure compatibility.

7.5 Dashboard Synchronization Problems

Initially, clicking the “Run Tests” button did not provide visible feedback to users.

Solution

A background execution system, live status monitoring, auto-refresh mechanism, and real-time log updates were implemented to improve interactivity.

8. GitHub Repository Management

The project was managed using GitHub. Version control practices included:

- Multiple commits during development
- Feature-based updates
- Structured project organization
- Documentation management

GitHub improved project tracking and demonstrated professional software development practices.

9. Conclusion

The Intelligent Test Execution and Reporting Framework has successfully met its goals by applying automated software testing at various levels. The framework showed unit testing, integration testing, and system testing, including advanced features like flaky test detection, graphical visualization, and an interactive dashboard.

The project provided practical experience with software quality assurance, automation, debugging, and software engineering practices. It also emphasized the significance of structured testing methodologies and automation tools in contemporary software development environments.

In general, the project effectively integrated theoretical ideas of software testing with their practical implementation and provided a complete automated testing solution.